Reproducibility and Improvement Study of LITETime for Time Series Classification

Kiran Meenakshi Sundaram
School of Computer Science
University College Dublin
Ireland
kiran.meenakshisundaram@ucdconnect.ie

Jose Ramon Morera Campos
School of Computer Science
University College Dublin
Ireland
Jose.moreracampos@ucdconnect.ie

Pelayo Garcia Alvarez
School of Computer Science
University College Dublin
Ireland
pelayo.garciaalvarez@ucdconnect.ie

***Abstract*—Time Series Classification (TSC) is a fundamental task in machine learning with applications in healthcare monitoring, industrial automation, activity recognition, and sensor analytics. Deep learning architectures such as InceptionTime have established strong performance benchmarks for TSC but often require large computational resources and extended training time.**

LITETime was introduced as a lightweight deep learning architecture designed to achieve competitive classification accuracy while significantly reducing training cost, parameter count, and energy consumption. These characteristics make LITETime particularly attractive for practical deployment in resource-constrained environments.

In this study, we independently reproduce the published LITETime results and evaluate targeted performance improvements under controlled experimental conditions. Reproduction experiments confirm the reliability of the original architecture, achieving performance comparable to reported benchmarks. Improvement experiments investigate optimization strategies including AdamW optimization, normalization methods, differenced input channels, and batch size variation.

Results demonstrate that meaningful performance improvements can be achieved through preprocessing and optimization strategies without modifying the underlying architecture. The strongest improvement was obtained using differenced input channels combined with global normalization. Ensemble analysis further confirms diminishing returns beyond moderate ensemble sizes. These findings highlight the robustness and efficiency of the LITETime framework and reinforce its suitability as an efficient deep learning solution for time series classification.

***Index Terms*—Time Series Classification, Deep Learning, Reproducibility, Lightweight Neural Networks, Ensemble Learning**

I. Introduction

Time series classification (TSC) is a critical task in modern machine learning, with applications across numerous domains including healthcare diagnostics, industrial monitoring, motion recognition, financial forecasting, and environmental sensing. In many of these applications, sequential observations must be analyzed to detect patterns that evolve over time. Unlike static tabular data, time series data contains temporal dependencies that must be modeled effectively to achieve reliable classification performance.

Traditional approaches to time series classification relied heavily on distance-based methods such as Dynamic Time Warping (DTW) combined with nearest-neighbor classifiers. These approaches achieved strong baseline performance but often required careful tuning of distance metrics and were computationally expensive for large datasets. Feature-based approaches such as shapelets and interval-based methods were later introduced to improve interpretability and classification efficiency.

In recent years, deep learning architectures have significantly advanced the state-of-the-art in time series classification. Models such as Fully Convolutional Networks (FCN), Residual Networks (ResNet), and InceptionTime have demonstrated strong predictive performance across large benchmark datasets. However, many of these architectures require substantial computational resources, large numbers of trainable parameters, and long training durations. These limitations can restrict their deployment in environments where computational power, memory, or energy availability is limited.

To address these challenges, the LITE architecture (Light Inception with boosTing tEchniques) was introduced as a lightweight deep learning model designed to reduce computational complexity while maintaining competitive accuracy. LITETime extends this concept by combining multiple LITE models into an ensemble framework, improving robustness and predictive stability. Compared to larger architectures such as InceptionTime, LITETime achieves competitive classification accuracy while requiring fewer parameters, faster training time, and reduced energy consumption. This balance between accuracy and efficiency positions LITETime as a strong lightweight alternative to traditional deep learning models for time series classification. These characteristics position LITETime as a strong lightweight alternative within deep learning-based time series classification.

The primary objective of this project is to reproduce the published LITETime results in an independent experimental environment and verify the reliability of reported findings. Reproducibility is an essential aspect of scientific research, ensuring that experimental results remain consistent across different hardware environments, software versions, and implementation settings.

A secondary objective is to evaluate targeted improvement strategies that enhance model performance without significantly modifying the architecture. Rather than introducing entirely new model structures, this study focuses on optimization techniques,

normalization strategies, and feature augmentation methods that improve learning efficiency while preserving architectural simplicity.

The major contributions of this study include:

- Independent reproduction of LITETime results using publicly available datasets.
- Development of a reproducible experimental pipeline supporting multiple configurations.
- Release of an open-source repository with a comprehensive `README.md` providing step-by-step reproduction instructions.
- Evaluation of targeted improvement strategies including optimizer selection, normalization methods, and input feature engineering.
- Analysis of ensemble size effects on predictive performance.
- Comprehensive study and evaluation of the base LITE architecture using the UCR archive.
- Comprehensive visualization and interpretation of experimental outcomes.

These contributions provide insights into the reliability, efficiency, and practical applicability of lightweight deep learning architectures for time series classification tasks.

LITETime has emerged as a strong lightweight alternative to large deep learning architectures such as InceptionTime. While achieving comparable classification accuracy, LITETime significantly reduces parameter count, training time, and energy consumption. These efficiency advantages make LITETime particularly suitable for deployment in real-world environments where computational resources are limited.

II. Motivation and Impact

The selection of LITETime for this project was motivated by the growing need for efficient deep learning models capable of handling large-scale time series classification tasks. Many state-of-the-art deep learning architectures achieve strong predictive performance but require significant computational resources, making them difficult to deploy in real-world environments with limited hardware capacity.

LITETime represents an important advancement in lightweight deep learning design. By combining trainable convolutional filters with hybrid handcrafted filters, the architecture achieves competitive accuracy while maintaining low computational complexity. This makes LITETime particularly suitable for applications involving embedded devices, edge computing platforms, and real-time monitoring systems.

Another key motivation for this project was the importance of reproducibility in machine learning research. Reproducibility ensures that published results remain valid across independent implementations and computing environments. By reproducing the LITETime results and extending the experiments through systematic optimization, this study contributes to improving transparency and reliability in time series classification research.

From a practical perspective, this project provides insights into how lightweight architectures can be optimized without requiring significant architectural redesign. The experimental findings demonstrate that preprocessing strategies, optimizer selection, and feature augmentation can meaningfully improve predictive performance.

The impact of this work extends to both academic research and real-world applications. Researchers can benefit from reproducible experimental pipelines, while practitioners can use the findings to deploy efficient models in resource-constrained environments.

III. Related Work

Time series classification has evolved significantly over the past two decades, transitioning from traditional distance-based techniques to modern deep learning architectures. Early approaches primarily relied on distance-based similarity measures such as Dynamic Time Warping (DTW) combined with nearest-neighbor classification. While these approaches were effective across many benchmark datasets, their computational complexity limited scalability to larger datasets.

Feature-based methods were later introduced to address these limitations. Shapelet-based techniques, for example, identify discriminative subsequences that capture class-specific temporal patterns. Interval-based and dictionary-based transformations further improved classification accuracy while reducing sensitivity to noise. However, these approaches often required manual feature engineering or computationally expensive search procedures.

Deep learning methods have recently become dominant in time series classification due to their ability to automatically extract hierarchical temporal features. Fully Convolutional Networks (FCN) demonstrated that convolutional architectures could effectively model temporal dependencies without manual feature design. Residual Networks (ResNet) extended this idea by introducing skip connections, allowing deeper architectures to be trained without degradation.

InceptionTime further improved performance by using multi-scale convolutional filters and ensemble learning. This architecture achieved strong performance across many datasets but introduced additional computational overhead due to increased model complexity and training time.

More recent lightweight approaches such as ROCKET and MiniROCKET demonstrated that efficient convolutional kernels can achieve strong classification performance while maintaining computational efficiency. These approaches highlighted the importance of balancing accuracy and efficiency in time series classification.

The LITE architecture was introduced to address this balance directly. By combining trainable convolutional filters with fixed hybrid filters and separable convolution operations, LITE reduces parameter count while preserving feature diversity. LITETime extends this architecture into an ensemble framework, improving robustness and predictive consistency. These characteristics make LITETime a compelling candidate for efficient deep learning-based time series classification.

IV. Architecture Overview

The LITE architecture is designed to capture temporal patterns efficiently using a combination of trainable and predefined convolutional filters. Unlike traditional deep learning models that rely exclusively on trainable filters, LITE introduces hybrid convolutional branches that enhance feature diversity while reducing computational overhead.

The architecture begins with a hybrid inception module. This module applies multiple convolutional operations using different kernel sizes, enabling the model to capture both short-term and long-term temporal dependencies. In addition to trainable filters, the hybrid branch includes fixed convolutional filters designed to detect alternating patterns and characteristic temporal changes.

The combination of trainable convolutional filters and fixed hybrid filters enables the model to capture multi-scale temporal patterns while maintaining a low parameter count. This design contributes to the strong efficiency advantages observed in comparison with deeper architectures.

Outputs from these parallel convolutional branches are concatenated and passed through batch normalization and nonlinear activation functions. This combination improves feature stability and reduces sensitivity to input scale variations.

Following the inception stage, separable convolution layers are applied to further refine extracted features. Separable convolutions reduce the number of parameters compared to standard convolutions while maintaining strong representational capacity. Dilated convolutions are also used to increase the receptive field without increasing computational complexity.

A global average pooling layer aggregates temporal information into a compact representation, which is then passed to a fully connected softmax classification layer. This structure enables efficient classification while maintaining low parameter overhead. It is worth noting that the total parameter count of the LITE model varies slightly depending on the number of classes in the target dataset—a detail not explicitly discussed in the original paper—though the resulting impact on architectural complexity remains marginal.

LITETime extends the LITE architecture by training multiple independent models and combining their outputs through averaging. Ensemble learning improves robustness and reduces sensitivity to random initialization.

V. Methodology

A. Datasets

All experiments were conducted using datasets from the UCR Time Series Classification Archive, which provides a standardized benchmark collection widely used in time series classification research. These datasets span multiple domains including biomedical signals, sensor readings, motion tracking, and simulated measurements.

Each dataset contains predefined training and testing splits, ensuring consistent evaluation across different studies. This standardization supports direct comparison with previously published results.

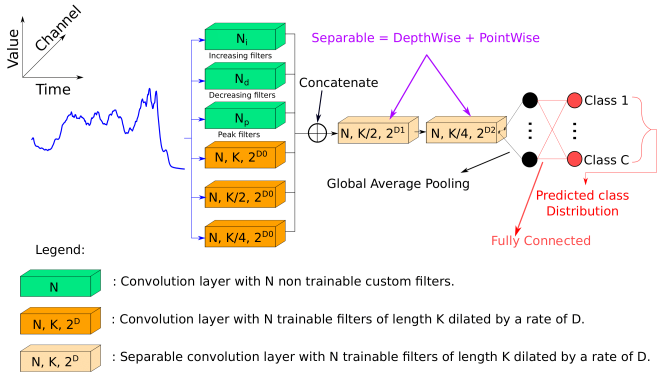

Fig. 1. Overview of the LITE architecture. The hybrid convolution structure combines trainable and fixed filters to capture multi-scale temporal dependencies while maintaining computational efficiency.

To enable repeated experimentation within limited computational resources, a representative subset of 30 datasets was selected. These were selected to maintain diversity in dataset size, sequence length, and number of classes while significantly reducing computational requirements. This fast subset preserves diversity across the archive while reducing the total training time for a five-model LITETime ensemble to approximately 5 hours, compared to over 20 hours required for the full UCR archive.

B. Data Preprocessing

All datasets were loaded using standardized utilities designed to ensure consistent formatting across experiments. Raw time series were reshaped into channel-last format to match the expected input structure of convolutional neural networks implemented in TensorFlow.

Normalization techniques were applied to improve training stability. In baseline experiments, standard normalization procedures were used. Additional improvement experiments evaluated alternative normalization strategies, including global z-normalization and global min-max normalization.

An additional preprocessing strategy involved computing differenced input channels. This technique calculates the temporal difference between consecutive values and appends the result as an additional input channel. The differenced channel provides explicit information about local temporal changes, improving the model's ability to detect dynamic patterns.

C. Implementation Details

The experimental pipeline was implemented using TensorFlow with high-performance GPU acceleration. To maximize throughput on modern hardware, we leveraged several low-level optimizations. Mixed precision training was employed to utilize Tensor Cores, significantly improving computational efficiency. Furthermore, XLA (Accelerated Linear Algebra) compilation was enabled via the `jit_compile` flag to optimize the execution graph. Hardware-level tuning was performed by enabling cuDNN auto-tuning and configuring a dedicated GPU thread pool

to ensure optimal kernel execution. Data throughput was optimized using the `tf.data` API, incorporating automated caching and prefetching mechanisms to eliminate I/O bottlenecks and minimize idle GPU time.

Several optimization strategies were evaluated to determine their impact on classification performance. These strategies included:

- Adam optimizer as the baseline training method.
- AdamW optimizer for improved generalization.
- Batch size variation to evaluate runtime-performance trade-offs.
- Global normalization methods to stabilize training.
- Differenced input channel augmentation to enhance feature representation.

A significant challenge during the initial phase of the project was the high computational demand and limitations in the original source code. We observed that the provided repository was primarily designed for single-dataset execution and was misconfigured with a default of only 2 epochs, whereas the original study requires 15,000 epochs for full convergence. Consequently, we developed an automated pipeline to facilitate batch processing across the entire UCR archive and corrected the training configuration to match reported benchmarks.

Training the complete five-model ensemble for LITETime originally required approximately 30 hours on an NVIDIA RTX-series GPU. To improve development speed and enable faster iteration, considerable effort was dedicated to optimizing the implementation. This included algorithmic refinements, such as refactoring operations out of loops, and the introduction of a non-verbose, no-plot mode to reduce overhead. Furthermore, performance was enhanced by leveraging specialized CUDA libraries and systematically testing different hardware and software configurations.

These implementation choices were designed to improve reproducibility and ensure consistent evaluation across experiments.

D. Training Configuration

Training was conducted using a consistent configuration across all experiments. Models were trained using categorical cross-entropy loss with adaptive learning rate scheduling. Multiple training runs were performed to reduce the effect of stochastic variation.

Batch size variation experiments were conducted to analyze the trade-off between computational efficiency and model generalization. Smaller batch sizes typically improve generalization but increase runtime, while larger batch sizes improve computational efficiency at the cost of slightly reduced accuracy.

All experiments were executed using GPU-enabled environments, ensuring efficient processing of multiple datasets.

E. Experimental Environment

All experiments were conducted using GPU-accelerated hardware to ensure efficient model training. The primary experimental environment consisted of:

- NVIDIA RTX-series GPU
- Python 3.10
- TensorFlow 2.x framework
- CUDA-enabled execution environment

Mixed precision training was enabled to improve computational efficiency and reduce memory usage. The execution environment was further tuned by configuring dedicated GPU thread pools and enabling `tf.data` auto-tuning to adapt to available CPU and memory resources. All software dependencies were controlled to ensure reproducibility across experimental runs.

VI. Reproducibility Study

The first stage of this project focused on reproducing the original LITETime results. Reproducibility is a critical component of reliable scientific research, ensuring that experimental findings remain valid across different environments and configurations.

Baseline experiments were executed using the original LITETime configuration with minimal modification. It was observed that the original repository did not implement fixed randomness (seeding) mechanisms. However, we assessed that for aggregate performance metrics over the UCR archive—and particularly for the LITETime ensemble framework across five independent runs—the architecture remains sufficiently reproducible to support reliable comparisons. Results were evaluated across multiple datasets to verify consistency with previously published performance metrics.

Reproduced results demonstrated performance values closely matching those reported in the original study. Minor variations were observed due to differences in hardware configuration, software versions, and floating-point precision behavior.

Table I summarizes the comparison between reported results and reproduced outcomes.

TABLE I
COMPARISON OF REPORTED AND REPRODUCED PERFORMANCE AND EFFICIENCY METRICS ACROSS THE FULL 128-DATASET UCR ARCHIVE

Model	Metric	Reported	Reproduced	Difference
LITE (Base)	Accuracy	0.8304	0.8316	+0.0012
	Training Time (s)	145,267	20,415	−124,852
	Energy (kWh)	0.6886	0.482	−0.2066
	CO_2 (kg)*	0.2928	0.1402	−0.1526
LITETime (Ensemble)	Accuracy	0.8430	0.8449	+0.0019

The small differences are within expected variation due to differences in computing environments.

It is worth noting that the original paper reported CO_2 emissions in grams, whereas the emissions tracking tool (CodeCarbon) used in their implementation actually reports values in kilograms*. Based on our reproduction results we can further confirm that the original paper unit, grams, was incorrect and should be kg instead. However, direct comparison of metrics across different studies is inherently limited by hardware variations. Our experiments utilized an NVIDIA RTX 4070, while the original study relied on an NVIDIA

GTX 1080. This architectural shift contributes to the observed results: significantly lower training times (86% reduction) and reduced total energy consumption due to the newer hardware and optimized implementation. Furthermore, CO_2 emissions depend heavily on the specific carbon intensity of the power grid in the region where the experiments were conducted. In contrast, energy consumption in kWh provides a more transferable metric of architectural efficiency. This observation further supports the efficiency advantages of LITETime and its impact on sustainable deep learning.

VII. Improvement Experiments

After validating reproducibility, targeted improvement experiments were conducted to evaluate whether performance gains could be achieved without modifying the underlying architecture. It is worth noting that these improvements were implemented after a complete execution run focused on verifying the original source repository behavior as closely as possible.

All improvement experiments were conducted using the fast subset of datasets. This subset enabled repeated experimentation while maintaining computational feasibility.

A. Optimizer Improvement: AdamW

The AdamW optimizer was evaluated as an alternative to the standard Adam optimizer. AdamW decouples weight decay from gradient updates, improving generalization and reducing overfitting.

Results showed a modest improvement in mean classification accuracy while maintaining nearly identical training time compared to the baseline configuration. Although the improvement was relatively small, it demonstrated that optimizer selection can influence predictive performance.

B. Batch Size Evaluation

Batch size selection involves a theoretical trade-off between computational efficiency and model generalization. In deep learning theory, smaller batch sizes are often preferred for their ability to introduce stochastic noise into the gradient updates, which can help the optimizer avoid sharp local minima and lead to better generalization. Conversely, larger batch sizes improve hardware utilization on modern GPUs by allowing for greater parallelization, which typically reduces total training time but may result in "flatter" minima and slightly lower predictive performance.

Our experimental results on the fast subset confirm these trends, as summarized in Table II. While the differences in mean accuracy across the configurations remain minimal, the impact on training duration is substantial. Batch size 32 achieved the highest mean accuracy for the base LITE model but required approximately 70% more training time than the 128-sample configuration. The LITETime ensemble framework demonstrates significant robustness, maintaining comparable performance across all tested batch sizes while allowing for faster throughput at higher configurations. Ultimately, these findings suggest that the batch size of 64 chosen in the original study represents an optimal balance between predictive accuracy and computational training cost.

TABLE II
Impact of Batch Size on Performance and Efficiency (Averages over Fast Subset)

Batch Size	LITE Acc. (Mean)	LITETime Acc.	Training Time (s)
32	0.8968	0.9018	270.86
64 (Baseline)	0.8950	0.9031	187.69
128	0.8943	0.9012	159.60

C. Normalization Strategies

Normalization methods significantly influenced model stability and predictive performance. Global z-normalization produced improved accuracy across most datasets, while global min-max normalization resulted in reduced performance.

These results indicate that z-score normalization provides more stable scaling across heterogeneous datasets compared to min-max normalization.

Among all evaluated strategies, the differenced input channel produced the strongest overall improvement. This technique computes the temporal difference between consecutive observations and appends the result as a secondary input channel, allowing the model to detect local rate-of-change patterns more effectively. Alternative preprocessing methods, such as moving averages, were considered unsuitable for classification as they often filter out discriminative high-frequency details essential for distinguishing between subtle class variations.

While this strategy provides substantial gains on average, our analysis reveals that it is not universally beneficial; for certain datasets, the inclusion of the differenced channel significantly degraded predictive performance. This suggests that future research should explore more sophisticated integration methods, such as weighted channel contributions or attention mechanisms, to adaptively modulate the influence of differenced features.

The addition of a secondary input channel slightly increases the model's complexity. As shown in Table III, adding the differenced channel to a sample dataset like "Coffee" results in a modest increase in both total and trainable parameters.

TABLE III
Parameter Count Comparison (Coffee Dataset)

Configuration	Dimensions	Total Parameters	Trainable
Base (1-Ch)	286×1	10,672	9,880
Diff. (2-Ch)	286×2	13,350	12,120

D. Best Configuration Summary

Across all tested configurations, the strongest overall performance was achieved using the following combination:

- Global z-normalization
- Differenced input channel

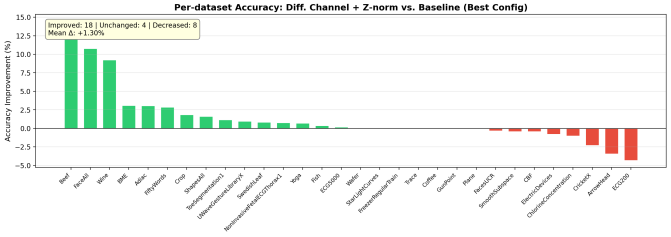

Fig. 2. Dataset-level improvement comparison demonstrating performance gains from optimization strategies including AdamW and differenced input channels.

- AdamW optimizer
- Ensemble size of 5 models

This configuration produced the highest mean accuracy while maintaining acceptable training efficiency.

VIII. Training Time and Efficiency Analysis

Training time and computational efficiency were analyzed to evaluate the practical feasibility of different configurations. Deep learning architectures frequently require substantial computational resources and extended training duration. Specifically, the original LITE implementation required approximately 0.62 days to train on the authors' hardware for a single member, which scales to roughly 3 days for the complete five-model ensemble.

Results demonstrated that certain improvement strategies increased accuracy without significantly increasing runtime, indicating efficient optimization.

Batch size variation produced noticeable runtime differences. Smaller batch sizes increased computational cost, while larger batch sizes reduced training time but slightly reduced predictive performance.

Additional analysis of runtime distribution confirmed that efficient configurations balanced predictive accuracy with computational cost.

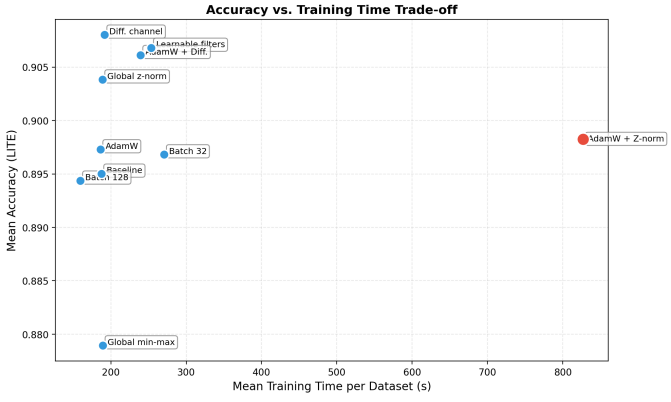

Fig. 3. Training time versus accuracy comparison across configurations. Efficient configurations achieved improved performance without significant runtime increase.

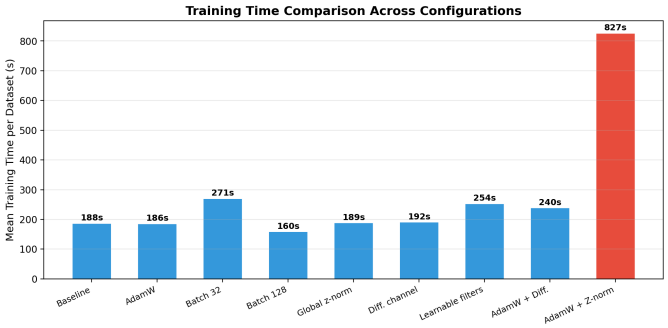

Fig. 4. Training time comparison across multiple configurations demonstrating the runtime trade-offs associated with different batch sizes.

IX. Accuracy Distribution Analysis

Accuracy distributions across datasets were evaluated to determine consistency of performance improvements.

The distribution analysis confirmed that performance improvements were not limited to individual datasets but were observed across multiple dataset types.

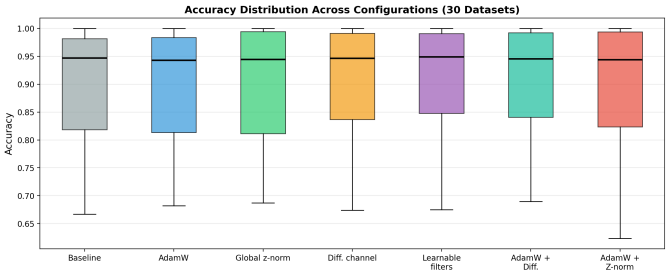

Fig. 5. Accuracy distribution across univariate datasets demonstrating performance consistency across configurations.

X. Ensemble Size Analysis

Ensemble learning is a core component of the LITETime framework. Multiple independently trained models are combined to improve predictive robustness and reduce variance caused by random initialization.

To evaluate the effect of ensemble size, experiments were conducted using ensemble sizes ranging from 1 to 10 models. Mean accuracy was measured across the fast subset datasets.

Results demonstrated that increasing ensemble size improves predictive performance up to a certain point. However, diminishing returns were observed beyond approximately five ensemble members.

This finding supports the use of moderate ensemble sizes as an effective compromise between performance improvement and computational cost. Excessively large ensembles introduce significant training overhead while producing only marginal performance gains.

XI. Discussion

The results obtained from this study provide several important insights into the behavior and performance

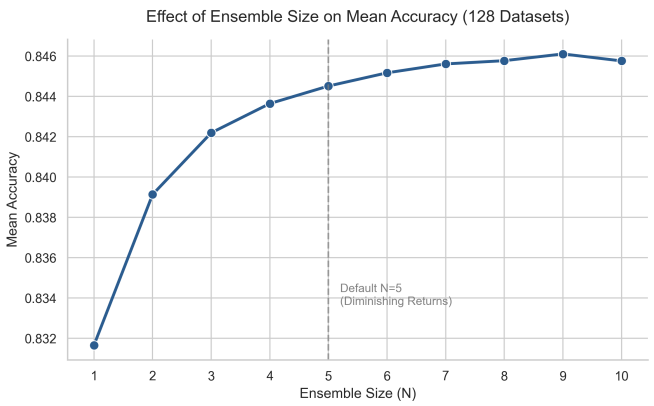

Fig. 6. Effect of ensemble size on classification accuracy. Performance improves with increasing ensemble size before stabilizing.

characteristics of lightweight deep learning models for time series classification.

First, the successful reproduction of published results confirms the reliability of the original LITETime architecture. Achieving comparable performance values demonstrates that the implementation pipeline is robust and consistent across independent environments.

Second, improvement experiments revealed that preprocessing and optimization strategies can significantly influence classification performance. Among all tested configurations, the differenced input channel produced the strongest performance gains. This finding emphasizes the importance of feature engineering in time series analysis.

Third, normalization strategies were found to strongly impact training stability. Global z-normalization produced consistent improvements across datasets, while global min-max normalization produced less stable results.

Fourth, ensemble analysis demonstrated diminishing returns beyond moderate ensemble sizes. This observation suggests that selecting an optimal ensemble size is essential to balance predictive accuracy and computational efficiency.

Additionally, training time analysis confirmed that certain configurations achieve improved performance without substantially increasing computational cost. These results support the practical viability of lightweight architectures such as LITETime in real-world applications.

Overall, the findings demonstrate that meaningful improvements can be achieved through systematic optimization without requiring significant architectural redesign.

XII. Limitations and Future Work

Although this study produced valuable insights, several limitations should be considered.

First, improvement experiments were conducted using a representative subset of datasets rather than the full dataset archive. While the subset preserved dataset diversity, performance trends may vary slightly across larger collections.

Third, the scope of experimental variations was constrained by available time and computational resources, which limited the extent of hyperparameter tuning and alternative architectural evaluations.

Future research could explore more sophisticated ensemble strategies beyond the currently implemented probability averaging. Investigating rank-based voting, weighted ensembling based on validation performance, or secondary stacking models represents an interesting direction for further enhancing classification accuracy. Additionally, further exploration of energy-efficient model design remains an important direction for continued research.

XIII. Conclusion

This study successfully reproduced the published LITETime results and validated the reliability of the original architecture. Reproduced performance closely matched reported benchmarks, confirming the reproducibility of the model across independent environments.

Improvement experiments demonstrated that performance gains can be achieved through targeted optimization strategies without modifying the core architecture. Among the evaluated methods, differenced input channels and global normalization provided the strongest improvements.

Ensemble analysis confirmed that moderate ensemble sizes offer an effective balance between performance and computational efficiency. Further evaluation demonstrated the adaptability and efficiency of LITE-based architectures across the UCR archive.

Overall, the findings highlight the effectiveness of lightweight deep learning architectures for time series classification and reinforce the importance of reproducible experimentation in machine learning research. These findings demonstrate that lightweight deep learning architectures such as LITETime provide an effective balance between predictive performance and computational efficiency, making them suitable for scalable real-world deployment.

Work Plan

The project was completed over an eight-week period with structured milestones to ensure steady progress and timely completion.

TABLE IV
Project Timeline

Phase	Duration
Paper selection and review	Week 1
Environment setup and dataset preparation	Week 2
Baseline reproduction	Weeks 3–4
Implementation optimization	Week 5
Improvement experiments	Weeks 6–7
Visualization, analysis and report writing	Week 8

Team Contributions

The project was completed collaboratively by all team members.

Kiran Meenakshi Sundaram contributed to dataset preparation, repository development, execution of baseline

experiments, generation of visualizations, and report preparation.

Jose Ramon Morera Campos contributed to implementation refinement, optimizer experiments, normalization studies, and evaluation of batch size configurations.

Pelayo Garcia Alvarez contributed to the study of the base LITE architecture, interpretation of experimental results across the UCR archive, visualization refinement, and literature review.

All members participated in experiment design, result validation, and report editing.

Data and Code Availability

All datasets used in this study are publicly available from the UCR Time Series Classification Archive.

The complete implementation, experiment configurations, and visualization scripts are available in the public project repository:

https://github.com/jose-r-morera/LITE

This open-source repository includes a detailed `README.md` providing step-by-step instructions to reproduce all results and visualizations presented in this report.

References

[1] A. Ismail-Fawaz, M. Devanne, S. Berretti, J. Weber, and G. Forestier, "Look Into the LITE in Deep Learning for Time Series Classification," *International Journal of Data Science and Analytics*, 2025.

[2] H. Dau *et al.*, "The UCR Time Series Classification Archive," 2018.

[3] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," *IJCNN*, 2017.

[4] Z. Wang *et al.*, "Residual networks for time series classification," 2017.

[5] H. Ismail-Fawaz *et al.*, "InceptionTime: Finding AlexNet for time series classification," *Data Mining and Knowledge Discovery*, 2020.

[6] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *ICLR*, 2019.

[7] A. Dempster, F. Petitjean, and G. Webb, "ROCKET: Exceptionally fast and accurate time series classification," 2020.

[8] A. Dempster, D. Schmidt, and G. Webb, "MiniROCKET: A very fast deterministic transform," 2021.

[9] J. Lines *et al.*, "Time series classification with shapelets," 2012.